ASCALON^IP

Getting Started and Integration Guide

Revision 2.2

Jul 21, 2025

tenstorrent

The information contained herein is for informational purposes only and is subject to change without notice. While every precaution has been taken in the preparation of this document, it may contain technical inaccuracies, omissions and typographical errors, and Tenstorrent is under no obligation to update or otherwise correct this information. Tenstorrent, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this document, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of Tenstorrent products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of Tenstorrent products are as set forth in a signed agreement between the parties or in the Tenstorrent Standard Terms and Conditions of Sale.

TRADEMARKS

A list of Tenstorrent's trademarks can be found at https://tenstorrent.com/trademarks**.** Other product names and links to external sites used in this publication are for identification purposes only and may be trademarks of their respective companies.

*Links to third party sites are provided for convenience and unless explicitly stated, Tenstorrent is not responsible for the contents of such linked sites and no endorsement is implied.

Table of Contents

Revision History *4*

Introduction *5*

Intended Audience **5**

Integration Pre-Requisites *6*

TT-Ascalon™ Cluster Package **6**

Documentation **8**

Compatible EDA Tools **8**

Setting-Up the Test Environment **8**

Using a Container **9**

Integration *10*

Standard Cells **10**

Memory Files **11**

Top-Level Files **13**

PLL Files **14**

Common Files **15**

Cluster IO **15**

Overview 15

System Memory Map **15**

PLL **16**

PLL Block 16

Interface Details 16

PLL Swapping 20

Temperature Integrations **21**

Interface Details 22

Swapping the Temperature Hub 26

Case 2: Swapping Controller Sequences 26

rv_temp_sens_model_generic 26

CPL Firmware *27*

CPL External Interfaces *27*

Reset and Reset Qualifiers **27**

SMC AXI Interface **28**

Interrupts **29**

CPL Register Access **30**

CPL SRAM Offsets **31**

RCPU 33

Format of _PSS Object: 33

PLL Parameters Format 33

Features/Flows implemented by CPL **34**

Reset Workflow 35

Thermal Management 37

Testbench ***38***

Requirements **38**

Building the Testbench **38**

Running the Testbench **38**

Running a Testlist 38

Using LSF (Optional) **38**

Running an Individual Test **38**

Checking Test Results 39

Manually Running a Test Outside of Provided Flows 39

Compiling and Running a Custom C Test 39

Dumping Waveforms 39

Viewing Design Hierarchy and Waveforms 40

Tests ***41***

Integration Targets ***42***

Requirements **42**

Building Integration Targets **42**

Linting Integration Targets **42**

CDC Constraints **42**

Whisper ***43***

Linux **43**

RTOS **43**

Revision History

Date	Version	Changes
Jul 21, 2025	2.2	Added information about CPL firmware and integration Added clock information for memory tiles Updated Cluster IO to reference newly-packaged ascalon_io IPXACT/HTML files
Apr 28, 2025	2.1	Added driving information for libcells Added VC Static CDC information Updated environment requirements
Feb 20, 2025	2.0	Updated branding, formatting, and proofreading
Dec 9, 2024	1.1.6	Added spyglass lint target Updated package structure diagram
Dec 02, 2024	1.1.5	Formatting
Nov 18, 2024	1.1.4	Added Xcelium support information
Nov 08, 2024	1.1.3	Proofreading and formatting
Oct 13, 2024	1.1.2	Updated: Makefile instructions, filelist paths, testlist names, directory structure, block names Added: additional integration files, PLL, temperature hub, and temperature sensor integration information, information on building physical design and simulation targets, information on building optimized and debug targets, information on dumping testbench waveforms, check_env.sh.
Sep 12, 2024	1.1.1	Updated branding and formatting.
Aug 15, 2024	1.1.0	Updated: branding and formatting, primitive cell integration instructions, memory integration instructions, internally verified simulation/waveform tool versions, references to abbreviated directories to reflect new expanded name structures, references to testbench to reflect new design_verification top-level directory. Added: memory file integration information, Cluster IO integration information
Apr 15, 2024	1.0	First release.

Introduction

This guide describes using the TT-Ascalon™ Cluster package, the included testbench, and included scripts and workflows.

Intended Audience

This document is intended for SoC designers and IP developers familiar with common EDA vendor tools and technology who intend to integrate the TT-Ascalon™ Cluster into a design.

Integration Pre-Requisites

TT-Ascalon™ Cluster Package

Figure 1 shows the TT-Ascalon™ Cluster directory structure after extracting the package. <$ROOT> is the directory that contains the extracted package.

Note: <$ROOT> also refers to the environment variable in the IP package files.

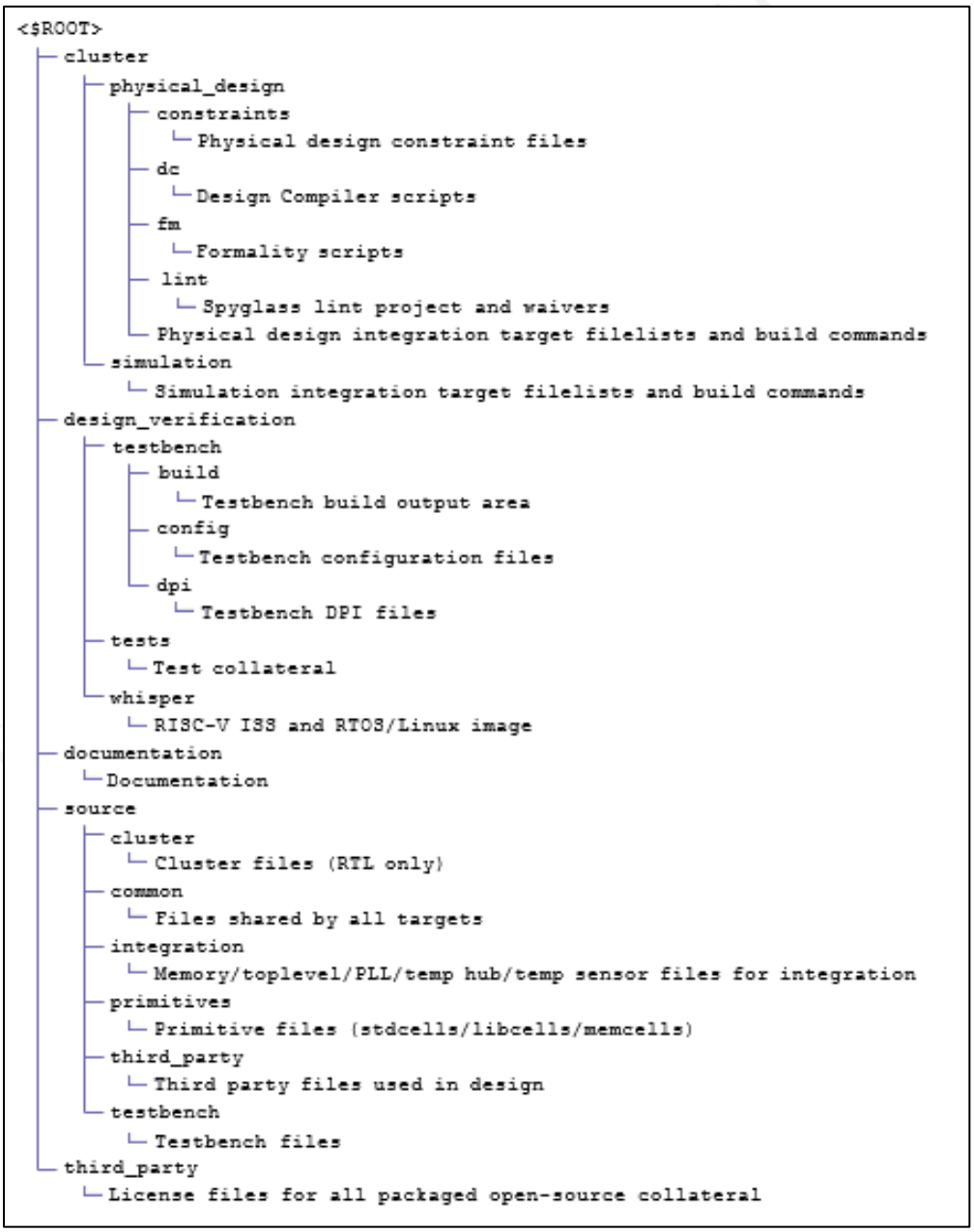


Figure 1 Ascalon Cluster Package Directory Tree

Documentation

TT-Ascalon™ Cluster includes the following documents:

- Getting Started and Integration Guide:
  - Describes the package contents, integration, testbench and the related tools for development.
  - Path: *documentation/ascalon_getting_started_and_integration.pdf*
- Physical Design Reference Guide:
  - Describes the physical design flows and using them with the TT-Ascalon™ Cluster package.
  - Path: *documentation/ascalon_physical_design_reference.pdf*
- Cluster Architecture Manual:
  - Describes the TT-Ascalon™ architecture, including system components, interfaces, supported RISC-V features, registers, and software sequences.
  - Path: *documentation/ascalon_arch_manual.pdf*
- Open-source licenses:
  - Describes the list of open-source licenses used in TT-Ascalon™ Cluster design and testbench environment, in the Markdown format.
  - Path: *documentation/open_source_licenses.pdf*

Compatible EDA Tools

Table 1 Verified and Compatible List of EDA Tools

Type	Version
Simulation	• Synopsys VCS U-2023.03-SP2-5 • Cadence Xcelium 24.03-s003
Waveform	Synopsys Verdi U-2023.03-SP2-5
Synthesis	Synopsys Design Compiler T-2022.03-SP4
Logical Equivalence	Synopsys Formality V-2023.12-SP2
RTL Lint	Synopsys Spyglass V-2023.12-SP1-1
CDC	Synopsys VC Static W-2024.09-SP1

Setting-Up the Test Environment

Perform the following steps to set up the test environment:

- Set the following environment variables:
  - Extract TT-Ascalon™ Cluster and set $ROOT to the extracted directory.
  - Install Synopsys VCS U-2023.03-SP2-5 and set $VCS_HOME to the installed directory.
  - Install Cadence Xcelium 24.03-s003 and set $XCELIUM_HOME to the installed directory.
  - Install Synopsys Verdi U-2023.03-SP2-5 and set $VERDI_HOME to the installed directory.

Note: Ensure to include the installed executables to $PATH for easy access and dumping waveforms.

- Install Python 3.6+ to build the testbench.
- Install Python 3.9+ to run the tests.

Note: The *$ROOT/design_verification/testbench/run_test.sh* script automatically creates and sources a Python venv at *$ROOT/design_verification/testbench/tenstorrent_tb_venv* with modules from *$ROOT/design_verification/testbench/requirements.txt*.

- Ensure that your Red Hat Enterprise LINUX platform contains:
    - GCC 12 and toolset binaries included in $PATH and sourced via *source scl_source enable gcc-toolset-12*.
    - GCC 12 libatomic library
    - GBLIC_2.26 header.
    - GLIBCXX_3.4.22 header.
    - CXXABI_1.3.9 header.

Tip: The header files are distributed as a part of the OS **glibc** installation.

    - **bc** package: Supports **bc** command.
    - **jq** package: Supports **jq** command. This command enables running the design verification test bench *design_verification/testbench/test_status.sh*
    - **libnsl:** Supports NIS library.
    - **numatl-libs:** Supports NUMA.
    - **perl:** Supports Perl.
    - **time:** Supports time command.
    - **boost1.78-devel**: Needed for testbench.

Using a Container

Using a container is an optional feature. You can build a container using the container file *$ROOT/design_verification/testbench/Containerfile*

To build and run a collateral inside the container:

1. Execute the following commands using containerization tools such as Docker or Podman.

```
docker build -t tt-ascalon-image -f
$ROOT/design_verification/testbench/Containerfile docker run --rm -it -v
$ROOT:/work -v $VCS_HOME:/vcs_home -v $XCELIUM_HOME:/xcelium_home -v
$VERDI_HOME:/verdi_home tt-ascalon-image
```

2. Start the container and run the following commands:

```
export ROOT=/work
export VCS_HOME=/vcs_home
export XCELIUM_HOME=/xcelium_home export VERDI_HOME=/verdi_home
export PATH=/opt/rh/gcc-toolset-
12/root/usr/bin:${VCS_HOME}/bin:${XCELIUM_HOME}/bin:${VERDI_HOME}/ bin:$$PATH
export SNSPSLMD_LICENSE_FILE=<license> (or alternative license env setup)
```

Integration

This section describes integrating the files from TT-Ascalon™ Cluster package.

Standard Cells

The unencrypted *source/primitives/tt_libcell_ *.sv* files provide the necessary standard cells. Ensure that both the following conditions are met, else use the behavioral versions of the cells:

- **SYNTHESIS** is defined.
- **NO_LIBCELL** is not defined.

Implement the cells in files from the table below for synthesis in the corresponding **ifdef** block for the **SYNTHESIS** define.

Most cells are instantiated as D1 (drive-strength 1).. However, in cases where timing is of concern, the synthesis tools to allow size swapping by adding set_size_only attribute to these cells.

- indication of dont_touch (size 1 expected) d0nt_<cell> in instance naming convention
- indication of size_only (size 1 instantiated, but allowed to resize) I_CLG_<cell> in instance naming convention
- <cell> is a cell type nd2, aoi22, nr2, etc

Note: The TT-Acsalon Cluster provides synchronizer wrapper modules for reference. These modules reference other primitive libcell modules and must not be modified.

Table 2 Standard Cells

File	Description
`tt_libcell_aoi22_quad.sv`	4-bit AOI
`tt_libcell_aoi22.sv`	1-bit AOI
`tt_libcell_clkand2.sv`	2-input clock AND gate
`tt_libcell_clkbuf.sv`	Clock buffer
`tt_libcell_clkgate.sv`	Clock gating cell
`tt_libcell_clkinv.sv`	Clock inverter
`tt_libcell_clkmux2.sv`	2-input clock MUX gate
`tt_libcell_clknand2.sv`	2-input clock NAND gate
`tt_libcell_clkor2.sv`	2-input clock OR gate
`tt_libcell_clkxor2.sv`	2-input clock XOR gate
`tt_libcell_dffrxq.sv`	D flip-flop with async active-low reset
`tt_libcell_dffsrxq.sv`	D flip-flop with async set and active-low reset
`tt_libcell_dff.sv`	Standard D flip-flop
`tt_libcell_dffsxq.sv`	D flip-flop with async set
`tt_libcell_fulladder.sv`	Full adder
`tt_libcell_halfadder.sv`	Half adder
`tt_libcell_lhcnqd4.sv`	D Latch with async clear
`tt_libcell_metastab_hardened_dffr.sv`	D flip-flop that is robust to metastability
`tt_libcell_metastab_hardened_dff.sv`	D flip-flop with active-low reset robust to metastability
`tt_libcell_nd2.sv`	2-input AND gate

tt_libcell_nd3.sv	3-input AND gate
tt_libcell_nd4.sv	4-input AND gate
tt_libcell_or2.sv	2-input OR
tt_libcell_stdand2.sv	2-input AND gate
tt_libcell_stdbuf.sv	Standard buffer
tt_libcell_stdinv.sv	Standard inverter
tt_libcell_stdmux2.sv	2-input multiplexer
tt_libcell_stdmux3.sv	3-input multiplexer
tt_libcell_stdnand2.sv	2-input NAND gate
tt_libcell_stdor2.sv	2-input OR gate
tt_libcell_sync2r_np.sv	2-stage synch using 2 clocks with async active-low reset
tt_libcell_sync2r.sv	2-stage synchronizer with async active-low reset
tt_libcell_sync2s.sv	2-stage synchronizer with async set
tt_libcell_sync2.sv	2-stage synchronizer
tt_libcell_sync3r.sv	3-stage synchronizer with async active-low reset
tt_libcell_sync3s.sv	3-stage synchronizer with async set
tt_libcell_sync3.sv	3-stage synchronizer
tt_libcell_sync4r.sv	4-stage synchronizer with async active-low reset
tt_libcell_sync4s.sv	4-stage synchronizer with async set
tt_libcell_sync4.sv	4-stage synchronizer
tt_libcell_xor2.sv	2-input XOR

Memory Files

Standalone files in the *source/integration* directory provide memories for each block are for reference. Some of these files have separate versions used specifically for the cluster physical design/simulation targets and the testbench target. Edit these files similarly to affect each target (**testbench**, **physical_design**, **simulation**).

Table 3 Memory Files

File	Description	Quantity	Configuration
tt_fe_mem_bimodal.sv	Table 0 of the branch direction predictor	2 banks	256x50b each. Operates on cluster clock (i_cl_clk).
tt_fe_mem_cic.sv	Stores a few pre-decoded bits of the instruction cache	16 instances	Each 128x78b instance holds data corresponding to 16B of instructions for 2 ways of 128 total indices in the instruction cache. Operates on cluster clock (i_cl_clk).
tt_fe_mem_icache.sv	Data portion of the instruction cache	32 instances	Each 256x78b instance holds 4 of the 8 ways of data for 16B of instructions of 32 of the 128 total

			indices in the instruction cache. Operates on cluster clock (i_cl_clk).
tt_fe_mem_itag.sv	Tag portion of the instruction cache	8 instances	Each 64x92b instance holds 2 of the 8 ways of tag information for 64 of the 128 total indices in the instruction cache. Operates on cluster clock (i_cl_clk).
tt_fe_mem_itlb_ram.sv	Instruction TLB	1 instance	64x96b. Operates on cluster clock (i_cl_clk).
tt_fe_mem_ittage.sv	Tables 1-5 of the branch target predictor	2 banks each, except where noted	• 128x78b each • 512x44b each • 512x44b each (4 banks) • 128x88b each 128x88b each • Operates on cluster clock (i_cl_cllk)
tt_fe_mem_tage.sv	Tables 1-6 of the branch direction predictor	Varies	• 512x44b each (2 banks) • 512x44b each (4 banks) 256x50b each (4 banks)
tt_ls_me1_dcdata.sv	Data portion of the Level 1 data cache	128 instances	Each 256x78b instance holds 2 of the 4 ways of data for a given word of memory. Operates on cluster clock (i_cl_clk).
tt_ls_me1_dctag.sv	Tag portion of the Level 1 data cache. 3 copies of the tag, one for each load pipeline	16 instances per tag copy	Each 256x50b instance holds 1 of the 4 ways of tag information for 256 of the 1024 total indices in the Level 1 data cache. Operates on cluster clock (i_cl_clk).
tt_ls_me2_l2tlb.sv	Level 2 TLB	12 instances each	• 256x72b each for the PA portion of the L2 TLB entries. 256x96b each for the VA portion of the L2 TLB entries. Operates on cluster clock (i_cl_clk).
	History table for the data prefetcher		Each256x132b instance holds a portion of the

tt_ls_me2_pht.sv		6 instances	prefetch history information. Operates on cluster clock (i_cl_clk).
tt_sc_slice_mem.sv	Cache regions, mapped to anything that is in DRAM region		8MB cache size. Operates on cluster clock (i_cl_clk).
tt_scb_mem.sv	Hold temporary data for DRAM, SP, and MMR regions	Varies	1R+1W (2 ported) and: • 12 entries x 148bits 48 entries x 160 bits. • Operates on cluster clock (i_cl_clk)
tt_trace_mem_sink.sv	Trace sink SRAM to capture signal trace information when signal tracing is enabled	8 instances	512x64 macros for a total of 32KB. Operates on cluster clock (i_cl_clk).
tt_TTCPLConfig_mem_0_ext.sv	CPL memory	4 instances	Single-port, no bit enable, 2048 entries, 72 bit entry size. Upper 8 bits of each 64-bit entry stores ECC. Operates on SOC clock (i_sc_clk).

Each file contains an instantiation of **tt_rv_mem_model** that is defined in a separate *standalone source/ integration/tt_rv_mem_model.sv file*. You can modify either file as needed to instantiate memories that meet the tile's memory requirement.

As an alternative to editing the of **tt_rv_mem_model** instantiation directly, the memory cells used by each tile are proived in the *primitives/tt_ascalon_mem_gen.sv* file and can be edited in-place. See that file for a full list of cells and descriptions in the comments.

Make the following pre-build changes to build and run with the technology swaps you made:

- Remove **+define+TT_BEHAVIORAL_MEM** from **$ROOT/design_verification/testbench/tt_testbench.f.**
- Remove **+define+TT_BEHAVIORAL_MEM** from **$ROOT/simulation/tt_cluster.f**.
- Remove **+define+TT_BEHAVIORAL_MEM** from **$ROOT/physical_design/tt_cluster.f**.

Top-Level Files

Top level files are provided in standalone files for integration and reference in the **source/integration** directory.

Table 4 Top-Level Files

File	Description
tt cluster.sv	**Cluster top-level file**
tt_cl_io.sv	**Cluster IO file**

tt_top_testbench.sv	**Testbench top-level file**
tt_cluster_harness_testbench.sv	**Testbench file that wraps the cluster**
tt_SC_fbchi_pkg.sv	**Package for shared cache interface with fabric CHI**
tt_SC_axi_pkg.sv	**Package for shared cache interface with AXI**

PLL Files

The TT-Ascalon Cluster includes the following files for integrating PLL, temperature sensor, and temperature hub modules in the **source/integration** directory except where noted.

Table 5 PLL Files

File	Description
tt_rv_pll_model_generic.sv	PLL file (located in **source/primitives**)
tt_clock_controller.sv	PLL clock controller used in **tt_cluster**
tt_cpl_top.sv	CPL wrapper file
tt_cpl_pkg.sv	CPL package file
tt_pll_controller.sv	PLL controller
tt_pll_pkg.sv	PLL package
tt_pll_wrapper.sv	PTL wrapper

Table 6 Temperature Sensor Files

File	Description
tt_rv_temp_sens_model_generic.sv	Temperature sensor file (located in source/primitives)
tt_rv_thub_model_generic.sv	Temperature hub file (located in source/ primitives)
tt_thub_controller.sv	Temperature hub controller
tt_thub_ctrl_pkg.sv	Temperature hub control package
tt_thub_top.sv	Wrapper file for temperature hub

Make the following pre-build changes to build and run with the technology swaps you made:

- Add **+define+CUSTOM_PLL**, **+define+CUSTOM_TEMP_SENS**, and **+define+CUSTOM_TEMP_HUB** to **$ROOT/design_verification/testbench/tt_testbench.f**.
- Add **+define+CUSTOM_PLL**, **+define+CUSTOM_TEMP_SENS**, and **+define+CUSTOM_TEMP_HUB** to **$ROOT/simulation/ tt_cluster.f**.

Add **+define+CUSTOM_PLL**, **+define+CUSTOM_TEMP_SENS**, and **+define+CUSTOM_TEMP_HUB** to **$ROOT/physical_design/tt_cluster.f**.

Common Files

The TT-Ascalon Cluster RTL includes extra standalone files that ensure all signals from the standalone memory files are visible from the top level and have the form **tt_common_part*.sv**. Do not alter these files when integrating.

Cluster IO

The TT-Ascalon Cluster RTL includes extra standalone files that ensure all signals from the standalone memory files are visible from the top level and have the form **tt_common_part*.sv**.

Overview

Table 7 Cluster IO

File	Description
`tt_rv_temp_sens_model_generic.sv`	Temperature sensor file (located in source/primitives)
`tt_rv_thub_model_generic.sv`	Temperature hub file (located in source/ primitives)
`tt_thub_controller.sv`	Temperature hub controller
`tt_thub_ctrl_pkg.sv`	Temperature hub control package
`tt_thub_top.sv`	Wrapper file for temperature hub

Cluster IO signals are enumerated in IPXACT and viewable HTML form in *documentation/ascalon_io.<html/ipxact>* files.

System Memory Map

The TT-Ascalon testbench uses JSON-based system memory map files when running tests. The default memory map JSON file is saved to
$ROOT/design_verification/testbench/config/memmap_json_path/memmap.json.
The system memory map contains one entry per region with the following fields:

- **type** – region type, which could be either memory or an IO device (e.g., **clint**, **htif**).
- **tag** – unique region name.
- **base** – region base address.
- **size** – region size in bytes.

Table 8 System Memory Map

Region	Description	Comments
`IO – M/S Level MMRs`	MMRs for 8 cores/SC/CPL etc.	
`IO – HTIF`	RISC-V host target interface for console emulation	
`IO – CLINT/ACLINT`	RISC-V CLINT/ACLINT devices for timer interrupts	
`IO – M/S Level Interrupt`	RISC-V AIA Interrupt Files	

Files		
IO – Trickbox	Platform specific region for custom DV traffic	MSIs, JTAG debug, entry/exit, etc.

PLL

PLL Block

The PLL0 and PLL1 blocks shown in Figure 2 are generic PLL models (**tt_rv_pll_model_generic**) that you can modify to integrate different vendor PLLs.

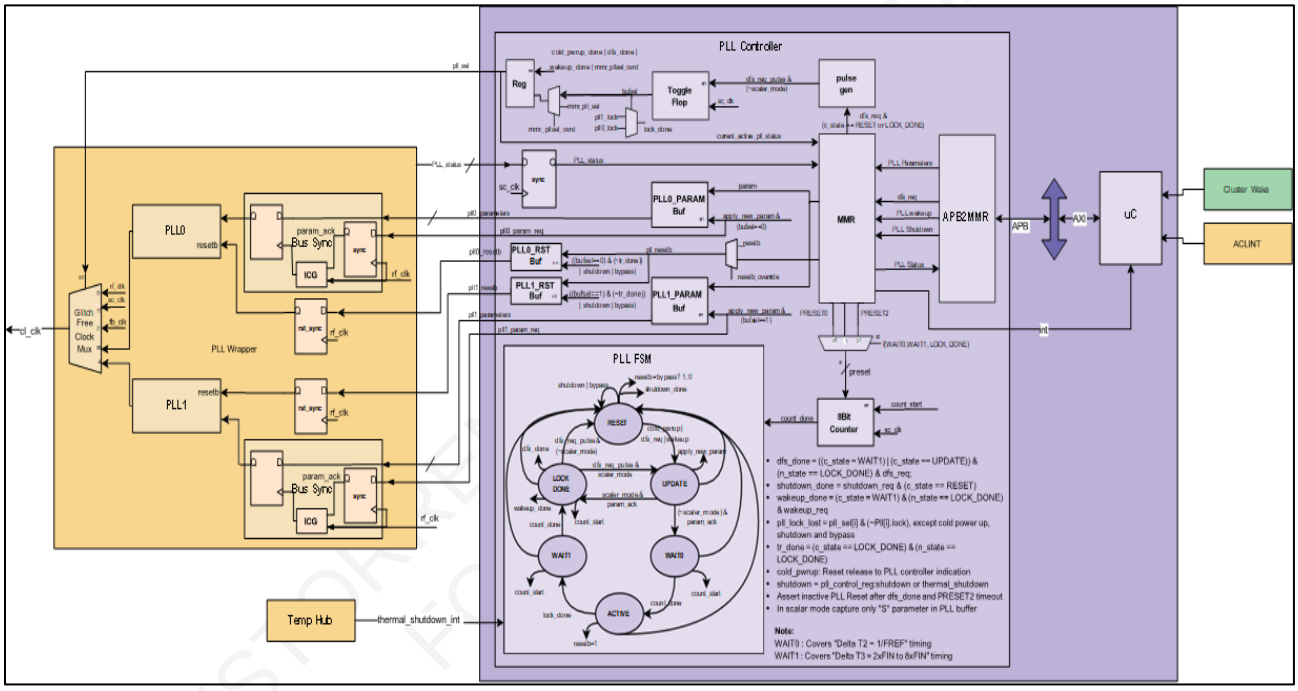


Figure 2 PLL Blocks

Interface Details

tt_rv_pll_model_generic

Table 9 tt_rv_pll_model_generic Interface

Interface Name	Dir	Bit Width	Default Value	ISO Value	Idle Value	Reset Value	Clock Domain	Reset Domain	Async/Desc. Sync	
rf_clk	In	1	1'b0				**rf_clk**			PLL reference clock
pll_clk	Out	1		1'b0	1'b0	1'b0	**cl_clk**			PLL clock output

reset_n	In	1	1'b0				**rf_clk**	**reset_n**	sync	Reference clock domain reset. Resets only in cold power-up.
pll_en	In	1	0				**rf_clk**	**reset_n**	sync	PLL enable control
pll_bypass	In	1	0				**rf_clk**	**reset_n**	sync	PLL bypass control
pll_parameters	In	***PLL_PARAM_ WIDTH***	0				**rf_clk**	**reset_n**	sync	PLL parameters and control signals
pll_status	In	***PLL_STATUS_ WIDTH***		0	0	0	**rf_clk**	**reset_n**	sync	PLL status info
tdr_select	In	*1*	1'b0				**tck**	**trstn**	sync	tdr select
tdr_mode	In	*1*	1'b0				**tck**	**trstn**	sync	tdr mode
tdr_clk	In	*1*	1'b0				**tck**	**trstn**	sync	tdr clk
tdr_rst_n	In	*1*	1'b0				**tck**	**trstn**	sync	tdr rst n
tdr_capture	In	*1*	1'b0				**tck**	**trstn**	sync	tdr capture
tdr_shift	In	*1*	1'b0				**tck**	**trstn**	sync	tdr shift
tdr_update	In	*1*	1'b0				**tck**	**trstn**	sync	tdr update
tdr_readback	In	*1*	1'b0				**tck**	**trstn**	sync	tdr readback
tdr_in	In	*1*	1'b0				**tck**	**trstn**	sync	tdr in
tdr_out	Out	*1*					**tck**	**trstn**	sync	tdr out
scan_mode	In	*1*	1'b0				**tck**		sync	scan mode
scan_clk	In	*1*	1'b0				**tck**		sync	scan clk
scan_en	In	*1*	1'b0				**tck**		sync	scan en
scan_in	In	***NUM_PLL_SCAN_CHAINS***	1'b0				**tck**		sync	scan in
scan_out	Out	***NUM_PLL_SCAN_CHAINS***					**tck**		sync	scan out

tt_pll_wrapper

Table 10 tt_pll_wrapper interface

Interface Name	Dir	Bit Width	Default Value	ISO Value	Idle Value	Reset Value	Clock Domain	Reset Domain	Async/ Sync	Desc.

sc_clk	In	1	1'b0				**sc_clk**			400MHz external clock; used for the entire CPL except PMNW master.
fb_clk	In	1	1'b0				**fb_clk**			1.8GHz external clock; used for fabric clock domain.
rf_clk	In	1	1'b0				**rf_clk**			100MHz external clock; used for reset controller.
cl_clk	Out	1		1'b0	1'b0	1'b0	**cl_clk**			2.7GHz internal clock; for PMNW master.
tb_cl_clk	In	1	1'b0				**tb_cl_clk**		-	TB generated 2.7GHz internal clock; supports verilator and Zebu.
rf_cold_ reset_n	In	1	1'b0				**rf_clk**	**rf_cold_reset_n**	sync	Reference clock domain reset. Resets only in cold power-up.
tb_pll_lock	In	1	1'b0				**rf_clk**	**rf_cold_reset_n**	-	TB generated PLL lock status; supports verilator and Zebu.
pll_sel	In	**clog2(*NUM_PLL*)**	1'b0				**rf_clk**	**rf_cold_reset_n**	async	PLL selection control
CPL_PLL_ Control	In	***[PLL_CPL_ CONTROL_WIDTH] [NUM_PLL]***	0				**rf_clk**	**rf_cold_reset_n**	async	PLL parameters and control signals
PLL_CPL_ Status	Out	***[PLL_CPL_ STATUS_WIDTH] [NUM_PLL]***		0	0	0	**rf_clk**	**rf_cold_reset_n**	async	PLL status info
tdr_select	In	1	1'b0				**tck**	**trstn**	sync	tdr select
tdr_mode	In	1	1'b0				**tck**	**trstn**	sync	tdr mode

tdr_clk	In	1	1'b0				**tck**	**trstn**	sync	tdr clk
tdr_rst_n	In	1	1'b0				**tck**	**trstn**	sync	tdr rst n
tdr_capture	In	1	1'b0				**tck**	**trstn**	sync	tdr capture
tdr_shift	In	1	1'b0				**tck**	**trstn**	sync	tdr shift
tdr_update	In	1	1'b0				**tck**	**trstn**	sync	tdr update
tdr_readback	In	1	1'b0				**tck**	**trstn**	sync	tdr readback
tdr_in	In	1	1'b0				**tck**	**trstn**	sync	tdr in
tdr_out	Out	NUM_PLL					**tck**	**trstn**	sync	tdr out
scan_mode	In	1	1'b0				**tck**		sync	scan mode
scan_clk	In	1	1'b0				**tck**		sync	scan clk
scan_en	In	1	1'b0				**tck**		sync	scan en
scan_in	In	NUM_PLL_ SCAN_CHAINS	1'b0				**tck**		sync	scan in
scan_out	Out	NUM_PLL_ SCAN_CHAINS] NUM_PLL								scan out

tt_pll_controller

Table 11 tt_pll_controller interface

Interface Name	Dir	Bit Width	Default Value	ISO Value	Idle Value	Reset Value	Clock Domain	Reset Domain	Async / Sync	Desc.
sc_clk	In	1	0				**sc_clk**			400MHz external clock; used for entire CPL except PMNW master
sc_cold_ reset_n	In	1	0				**sc_clk**		sync	SOC clock domain cold reset
force_ss_to_ ref_clock_n	In	1	0				**sc_clk**		async	Control to force all clocks to reference clock
pll_ interrupts_in	In	***NUM_PLL_INT_IN***	0						async	PLL input interrupts
pll_ interrupts_out	Out	***NUM_PLL_INT_OUT***		0	0	0			sync	PLL output interrupts
paddr	In	***APB_ADDR_WIDTH***	0				**sc_clk**	**sc_cold_reset_n**	sync	APB address bus
pprot	In	3	0				**sc_clk**	**sc_cold_reset_n**	sync	APB protection
psel	In		0				**sc_clk**	**sc_cold_reset_n**	sync	APB sel

penable	In		0				**sc_clk**	**sc_cold_reset_n**	sync	APB enable
pwrite	In		0				**sc_clk**	**sc_cold_reset_n**	sync	APB write
pwdata	In	APB_DATA_WIDTH	0				**sc_clk**	**sc_cold_reset_n**	sync	APB write data
pstrb	In	APB_DATA_WIDTH/8	0				**sc_clk**	**sc_cold_reset_n**	sync	APB strobe
pready	Out			0	0	0	**sc_clk**	**sc_cold_reset_n**	sync	APB ready
prdata	Out	APB_DATA_WIDTH		0	0	0	**sc_clk**	**sc_cold_reset_n**	sync	APB read data
pslverr	Out			0	0	0	**sc_clk**	**sc_cold_reset_n**	sync	APB slave error
pll_sel	Out	PLL_SEL_WIDTH		0	0	0	**sc_clk**	**sc_cold_reset_n**	sync	PLL selection control
PLL_CPL_Status	In	PLL_CPL_STATUS_WIDTH	0				**sc_clk**	**sc_cold_reset_n**	async	PLL status info
CPL_PLL_Control	Out	CPL_PLL_CTRL_WIDTH		0	0	0	**sc_clk**	**sc_cold_reset_n**	async	PLL parameters and control signals

PLL Swapping

This section describes two PLL swapping use cases.

Case 1: PLL Controller Sequences Matches Swapped PLL Requirements

- Update the following RTLs:
  - **source/primitives/tt_rv_pll_model_generic.sv**
  - **source/integration/tt_pll_pkg.svh**
- Update **source/primitives/tt_rv_pll_model_generic.sv**:
  - Instantiate the required PLL model under **ifdef CUSTOM_PLL**.
  - Define **CUSTOM_PLL** as part of the compile.
  - Integrate PLL interfaces with **tt_rv_pll_model_generic i**nterface:
    - > Connect **ref_clk** to the PLL reference clock interface and **pll_clk** to PLL clock out.
    - > Use the **pll_parameters** input interface (driven from the PLL controller MMR) to control PLL parameters. Max supported PLL parameters: 125 bits.
    - > Use the **`pll_status`** output interface (mapped to the PLL controller MMR) to capture the PLL status. x supported PLL status: 15 bits.
    - > Tie off unused control inputs.
    - > Open unused status outputs.
- Update **`source/integration/tt_pll_pkg.svh`**:
  - Configure the default frequency by defining local PLL parameters that define PoR values in accordance with the PLL requirement.

Case 2: Update PLL Controller Sequences for Swapped PLL

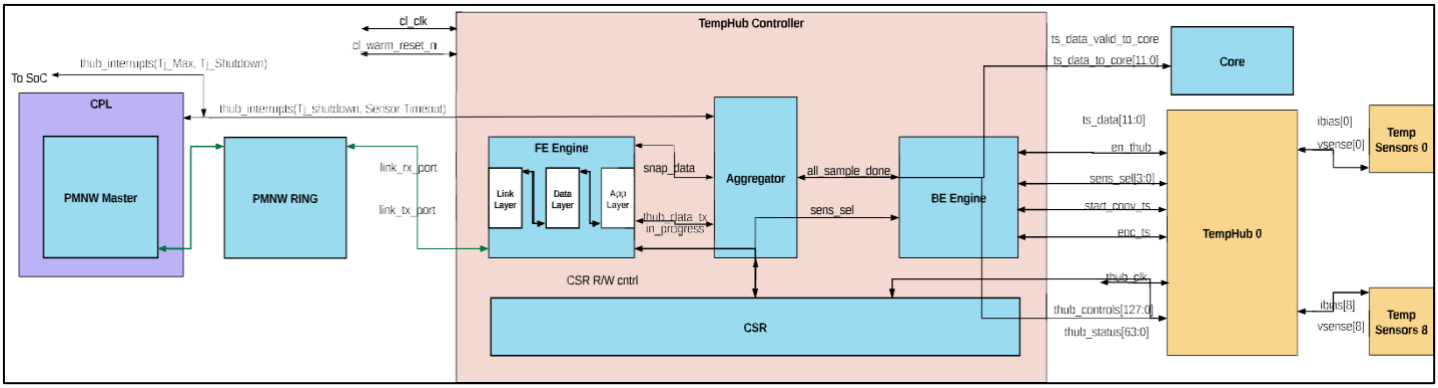
- Update the RTL:
  - **source/primitives/tt_rv_pll_model_generic.sv**
  - **source/integration/tt_pll_pkg.svh**
  - **source/integration/tt_pll_wrapper.sv**
  - **source/integration/tt_clock_controller.sv**
  - **source/integration/tt_cluster.sv**
  - **source/integration/tt_cpl_top.sv**
  - **source/integration/tt_pll_controller.sv**
- Update **source/integration/tt_pll_controller.sv**:
  - Develop the PLL controller per the PLL sequence requirements.
  - Generate the required MMRs (**cc_csr** module) and hook them to the APB interface.
  - Design requirements:
    > Drive **pll_sel** to select **rf_clk** when **force_ss_to_ref_clock_n** is set to **0**, else select the required PLL clock.
    > Set pll_inactive to **1** when PLLs are shutting down or waking up to prevent lock lost interrupt to CPL.
    > Update **pll_interrupts_in** and **pll_interrupts_out** based on interrupt mapping.
    > Take care of all CDC crossings between the **pll_controller** (in the **SC_CLK** domain) and **tt_pll_wrapper** (in the **rf_clk domain**).
- Update **source/integration/tt_pll_pkg.svh**:

Update/define required parameters per the **tt_pll_controller** and **tt_rv_pll_model_generic** changes.

- Update **`source/integration/tt_pll_wrapper.sv`**:
  - Update the number of PLL instantiations.
  - Update the **tt_rv_clk_mux_glitch_free** instantiation based on the number of PLL instantiations.
- Update **source/primitives/tt_rv_pll_model_generic.sv**:
  - PLL Instantiation
  - Integration updates
- Update **source/integration/tt_cpl_top.sv**, **source/integration/ tt_cluster.sv**, and **source/integration/tt_clock_controller.sv**:
  - Integration updates per the new interface additions in the **tt_pll_controller** and **tt_rv_pll_model_generic** modules.

Temperature Integrations

This section describes how to integrate the Temperature Hub, Temperature Sensor, and Temperature Controller.



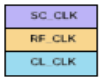


Figure 3 Temperature Integration Block Diagram

Interface Details

thub_controller

Table 12 thub_controller Interface

Interface Name	Dir	Bit Width	Default Value	ISO Value	Idle Value	Reset Value	Clock Domain	Reset Domain	Async / Sync	Desc.
i_cl_clk	In	1	1'b0				**cl_clk**			2.7GHz internal clock; for PMNW master
i_cl_warm_reset_n	In	1	1'b0				**cl_clk**		sync	Cluster clock domain reset. Resets both in cold and warm power-up.
i_link_rx_port	In	**link_t**	0				**cl_clk**	**cl_warm_reset_n**	sync	PMNW ring input.
o_link_tx_port	Out	**link_t**		0	0	0	**cl_clk**	**cl_warm_reset_n**	sync	PMNW ring output.
o_thub_interrupts	Out	***NUM_THUB_INT_OUT***		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Temperature Hub interrupts: **Tj_Max** and **Tj_Shutdown**
o_en_thub	Out	1		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Temperature Hub enable control.
o_sens_sel	Out	4		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Sensor selection control to Temperature Hub.

o_start_conv_ts	Out	1		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Conversion trigger to Temperature Hub.
i_eoc_ts	In	1	0				**cl_clk**	**cl_warm_reset_n**	async	End of conversion response from Temperature Hub.
i_ts_data	In	***SENS_DATA_WIDTH***	0				**cl_clk**	**cl_warm_reset_n**	async	Converted Temperature Sensor data.
o_thub_controls	Out	128		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Spare Temperature Hub controls from CSR for generic usage.
o_thub_controls_valid	**Out**	**1**		**0**	**0**	**0**	**cl_clk**	**cl_warm_reset_n**	**async**	Valid indication Temperature Hub controls.
i_thub_controls_ready	**In**	**1**	0				**cl_clk**	**cl_warm_reset_n**	**async**	Ready indication from Temperature Hub after sampling controls.
o_thub_fuse_controls	**Out**			**0**	**0**	**0**	**cl_clk**	**cl_warm_reset_n**	**async**	Fuse controls for Temperature Hub.
i_thub_status	**In**	**64**	0				**cl_clk**	**cl_warm_reset_n**	**async**	Spare Temperature Hub status to CSR for generic usage. Multibit sync assumed mutually exclusive individual status bits.

o_ts_data_to_core	**Out**	**SENS_DATA_WIDTH**		**0**	**0**	**0**	**cl_clk**	**cl_warm_reset_n**	**sync**	Aggregated Temperature Sensor data to core.
o_ts_data_valid_to_core	**Out**	**1**		**0**	**0**	**0**	**cl_clk**	**cl_warm_reset_n**	**sync**	Temperature Sensor data valid indication to core.

tt_rv_thub_model_generic

Table 13 tt_rv_thub_model_generic Interface

Interface Name	Dir	Bit Width	Default Value	ISO Value	Idle Value	Reset Value	Clock Domain	Reset Domain	Async/ Sync	Desc.
i_clk	In	1	0				**rf_clk**			100MHz external clock; used for Reset Controller.
i_reset_n	In	1	0				**cl_clk**		sync	Cluster clock domain reset. Resets both in cold and warm power-up.
i_en_thub	In	1	0				**cl_clk**	**cl_warm_reset_n**	async	Temperature Hub enable control.
i_sens_sel	In	***SENS_SEL_WIDTH***	0				**cl_clk**	**cl_warm_reset_n**	async	Sensor selection control to Temperature Hub.
i_start_conv_ts	In	1	0				**cl_clk**	**cl_warm_reset_n**	async	Conversion trigger to Temperature Hub.
o_eoc_ts	Out	1		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	End of conversion response from Temperature Hub.
o_ts_data	Out	***SENS_DATA_WIDTH***		0	0	0	**cl_clk**	**cl_warm_reset_n**	async	Converted Temperature Sensor data.

i_thub_controls	In	***THUB_CTRL_WIDTH***	0				**cl_clk**	**cl_warm_ reset_n**	async	Spare Temperature Hub controls from CSR; for generic usage.
i_thub_ controls_valid	In	1	0				**cl_clk**	**cl_warm_ reset_n**	async	Valid indication Temperature Hub controls.
o_thub_ controls_ready	Out	1		0	0	0	**cl_clk**	**cl_warm_ reset_n**	async	Ready indication from Temperature Hub after sampling controls.
i_thub_fuse_ controls	In	***THUB_FUSE_ CTRL_WIDTH***	0				**cl_clk**	**cl_warm_ reset_n**	async	Fuse controls for Temperature Hub.
o_thub_status	Out	64		0	0	0	**cl_clk**	**cl_warm_ reset_n**	async	Spare Temperature Hub status to CSR for generic usage. Multi-bit sync assumes mutually exclusive individual status bits.
i_scan_en	In	1	0							DFT scan enable.
i_scan_in	In	1	0						sync	DFT scan in.
o_scan_out	Out	1		0	0	0			sync	DFT scan out.
ibias_ts	In/ Out	***NUM_SENS_PER_ HUB***							sync	**ibias** from sensor.

vsense_ts	In/ Out	***NUM_SENS_PER_ HUB***								**vsense** from sensor.
test_out_ts	In/ Out	1								Analog test bus.
vol_ts	In/ Out	14								Voltage sensor input; measures voltage.

Swapping the Temperature Hub

This section describes two use cases when swapping the Temperature Hub.

Case 1: Matching Controller Sequences

- Update the **source/primitives/tt_rv_thub_model_generic.sv RTL**.
- Update **source/primitives/tt_rv_thub_model_generic.sv**:
  - Instantiate in `**ifdef CUSTOM_TEMP_HUB**.
  - Define **CUSTOM_TEMP_HUB** as part of the compile.
  - Integrate **THUB** interfaces with the **tt_rv_thub_model_generic** interface.
    - > Use the **thub_controls_to_hub** input interface (driven from the Temperature Hub controller MMR) to control Temperature Hub controls. Max supported PLL parameters: 127 bits
    - > Use the thub_status output interface (mapped to the Temperature Hub controller MMR) to capture the Temperature Hub status. Max supported PLL status: 63 bits
    - > Use the **thub_fuse_controls** input interface (driven from the Temperature Hub controller MMR) to control the Temperature Hub fuse. Max supported Temperature Hub fuses: 62 bits
    - > Tie off unused control inputs.
    - > Open the unused status output.

Case 2: Swapping Controller Sequences

- Update the following RTLs:
  - **source/primitives/tt_rv_thub_model_generic.sv**
  - **source/integration/tt_thub_top.sv**
  - **source/integration/tt_thub_controller.sv**
  - **source/integration/tt_thub_ctrl_pkg.svh**
- Update **source/integration/tt_thub_controller.sv** and **source/integration/tt_thub_ctrl_pkg.svh** based on the Temperature Hub requirements.
- Integrate the Temperature Hub in **source/primitives/tt_rv_thub_model_generic.sv**.

Update the interface changes in **source/integration/tt_thub_top.sv**.

rv_temp_sens_model_generic

tt_rv_temp_sens_model_generic consists of third-party Temperature Sensors. You can update this model based on the third-party IP, interface, and connectivity to the Temperature Hub.

CPL Firmware

CPL firmware is stored in **$ROOT/design_verification/testbench/config/cplfw_path** as a standalone ELF file.

CPL External Interfaces

Reset and Reset Qualifiers

CPL implements the reset controller logic. CPL has the following single bit reset control input pins, listed in Table 14.

Table 14 Single Bit Reset Control Input Pins

Pin	Polarity	Scope
i_cluster_cold_reset_n	Active Low	Reset all the blocks across all clock domains
I_cluster_warm_reset_n	Active Low	Reset blocks taking into consideration warm reset qualifiers. Please see entries for ***hold** signals below.
i_cluster_sram_hold	Active High	If set, CPL SRAM content is preserved on warm reset.
i_cluster_debug_hold	Active High	If set, the debug module (DM) is affected by warm reset.
i_cluster_critical_signal_hold	Active High	If set, Performance Counters, and Dfd (Design For Debug) blocks' registers are preserved across warm reset. **NOTE**: Dfd blocks description are outside the scope of this document.

SMC AXI Interface

CPL supports the AXI4 interface with a 64-bit data bus. The 32-bit address bus runs on the SOC clock (**i_sc_clk**).

The external System Management Controller (SMC) accesses CPL and the following Ascalon resources using this interface:

- CPL SRAM & Registers (see Table 10 in the *Ascalon Cluster Architecture Specification*)
- Ascalon MMRs
- Power Management Network (an intra-Ascalon network that accesses temperature hub/sensors and power management blocks in each of the 8 CPU cores)

Interrupts

CPL implements the following interrupts, as listed in Table 15.

1. **TjMax Interrupt** - A level-triggered output (pin name: **o_thub_tj_max**) to signal fatally high temperatures (outside of operational range) in the Ascalon cluster.
2. **SMC Interrupts** - 24-bit interrupt lines (pin name: **o_cluster_interrupts**) driven by CPL firmware and hardware. External micro-controllers (such as a System Management Controller) handle these events in accordance with system requirements. For brevity, we assume that SOC vendors use CPL FW image for reset, power, and thermal management. In such case not all interrupts need servicing by SMC. Some of the interrupts need to be masked, as indicated.

Table 15 CPL Interrupts

Position	Interrupt Name	Mask	Purpose
12:0, 17:14	Power Management Transitions	Yes	CPL FW provides APIs for power management transitions. All interrupts should be masked by SMC.
13	Tj Shutdown	No	A level-triggered interrupt to signal high temperature in the Ascalon cluster, which is still within operational range. SMC indicates the OS to save state and shutdown. CPL will bring down the frequency to **P-n**.
18	Config Done	No	CPL indication to SMC that ASC clusters are configured. In response, SMC supplies the boot code to fetch.
19	Ext DFS Done	No	CPL services the write frequency on update to **PllExtControlReg**. SMC reads the **PllInterruptsReg** for the request **Status**.
20:22	Reserved	N/A	N/A
23	Zkr	No	SMC updates the **SeedCSR** registers as per information in the doorbell **event3** register.

CPL Register Access

Error! Reference source not found. lists values to be driven on the ports of the SMC-AXI interface to access MMRs and CPL registers:

- **USER ID**: For successful access on the SMC AXI Interface, the User ID input (AWUSER and ARUSER) must be set to 0x3 (System Management Controller source ID).
- **ARSIZE/AWSIZE**: Ascalon supports both 32-bit and 64-bit registers. Set ARSIZE and AWSIZE accordingly
  - 0x2 for 32-bit register access
  - 0x3 for 64-bit register access
- **ADDRESS**: Machine-level MMRs and CPL register accesses are required to support CPL functional features. Set Address [26:19] as {2'b01, 4-bit cluster id, 2'b10}. For resets of address bits, refer to Table 16
- Table 16.

Table 16 CPL Resource ID Encoding

Unit: CPL Resources	Address Bits							
	18	17	16	15	14	13	12	11:0
Watch Dog Timer	0	0	0	0	0	0	0	Offset (4KB Size)
CPL Debug Module	0	0	0	0	0	0	1	Offset (4KB Size)
Reset Unit	0	0	0	0	0	1	0	Offset (4KB Size)
PLL Interface	0	0	0	0	0	1	1	Offset (4KB Size)
CPL Registers	0	0	0	1	0	0	Offset (8KB Size)	
Inbound Filters	0	0	0	1	1	0	1	Offset (4KB Size)
Outbound Filters	0	0	0	1	1	1	0	Offset (4KB Size)
SRAM	1	0	Offset (128 KB)					
Power Management Network	1	1	1	Offset (64 KB Size)				

CPL SRAM Offsets

CPL uses regions in CPL SRAM to get **initialization and control data** from external entities such as RCPU. Table 17 describes CPL SRAM offsets and SRAM addresses. Table 18 describes the init format including offsets and default values.

Table 17 CPL SRAM Offsets

Regions	Size in KB	Offset	AXI Address
CPL FW	32	0	0x2140000
Init (See Table 18)	4	32	0x2148000
Fuse and Config (See RCPU)	8	36	0x2149000

Table 18 Init Format

Object	Offset	Default Values
Number of P-states	2148004	This variable determines the number of valid **s** objects. Limit to 8 **P_states**.
PTJ_Shutdown, P1, P2,… _PSS Object	214800c	Support 8 **_PSS** objects. Each **_PSS** object is 32 bytes wide.
Pll parameters for each P-state	2148114	Support 8 Pll parameters (one for each P-State). Each parameter is 40 bytes wide.
C State Limit (supported)	21481b8	ASC supports up to C4. External Agent to limit max C-state as per state of rest of the SOC/Chiplet.
Curr. C State Limit	21481c0	
Current Highest Frequency	21481c8	ASC Does not have VR Ctrl. External Agent to limit frequency as per current voltage

RCPU

RCPU provides the following values for **Fuse and Config**:

1. CrClusterFuseCsr (offset 0x0)
2. ResetVector (offset 0x8)

Format of _PSS Object:

```
struct pstate {
       uint32_t frequency_mhz;
       uint32_t power_mw;
       uint32_t transition_latency_us; /* Microseconds */
       uint32_t bus_master_latency_us; /* Microseconds */
       uint64_t control; → Pstate NUmber
       uint64_t status; → Current Pstate
};
```

PLL Parameters Format

```
struct pstate_pll_parameters {
  struct pstate_pll_parameters_valid valid;
  struct pstate_pll_parameters_value value;
};
struct pstate_pll_parameters_value {
  uint32_t pll_parameters_0;
  uint32_t pll_parameters_1;
  uint32_t pll_parameters_2;
  uint32_t pll_parameters_3;
};
struct pstate_pll_parameters_valid {
  uint8_t pll_parameters_0_valid;
  uint8_t pll_parameters_1_valid;
  uint8_t pll_parameters_2_valid;
  uint8_t pll_parameters_3_valid;
};
```

Features/Flows implemented by CPL

CPL implements and exposes hardware and firmware interfaces for the following features:

- Reset Workflow to reset the Ascalon cluster and initiate code fetch from the cores
- Thermal Management
    - TjMax
    - TjShutdown

This document outlines pseudo-code and Interrupt Service Routines (ISRs) for features supported by CPL. It will be periodically updated as additional features, such as low power entry/exit and dynamic frequency scaling, are integrated into the design.

Reset Workflow

Table 19 describes the sequential CPL Reset Workflow for Cluster Pin Control, SMC Programming, and CPU Execution.

Table 19 CPL Reset Workflow

Steps	Register / Pin Information
Cluster Pin Control	
SMC activates SOC clock and Fabric clock (Reference clock is stable)	Pin: i_ss_clk, i_fb_clk. i_rf_clk
SMC set force_ss_to _ref_clock_n to '0' to run all clocks with REF clock o Use to meet timing on reset paths. o All cluster FB and SC AXI i/f are fenced when force_ss_to _ref_clock_n set '0'.	Pin: i_force_ss_to_ref_clock_n
SMC asserts cluster_cold_reset_n by reset unit in SMC o cluster_cold_reset_n asserts cpl_warm_reset_n and deassert dmactive to initialize the system	Pin: i_cluster_cold_reset_n
SMC drives chiplet ID	Pin: i_cluster_id
SMC deasserts cluster_cold_reset_n o All domains except cl_*_reset_n, rf_warm_reset_n will be out of reset	Pin: i_cluster_cold_reset_n
SMC set force_ss_to _ref_clock_n to '1' to switch fb and sc clocks to its original source	Pin: i_force_ss_to _ref_clock_n

Steps	Register / Pin Information
SMC Programming – 1	
SMC downloads firmware and Fuse config to CPL SRAM through PM-AXI	Fuse Config Offset in SRAM: @ 0x9000: Layout of Data this offset: Offset: 0x9000: CrClusterFuseCsr (please see Config Register access) Offset: 0x9008: Reset Vector (All cores in ASC cluster share the reset vector)
SMC configures CPL reset vector	reset_vector[0] : Defined by Customer.
SMC programs CPL reset ctrl to start execution	Reset_ctrl : 0x1_0020, bit[0] to 1 (reset_ctrl)
CPL Execution – 1	
CPL sends Config Done Interrupt to SMC	CPL RC register(Offset 0xc): RstCtrlCplCLNoFetchReg[ConfigDone] CPL sets bit[31] to 1
SMC Programming – 2	
After SMC receives all cluster config done, SMC sends start execution of core to CPL	CPL RC register(Offset 0xc): RstCtrlCplCLNoFetchReg[ClusterCoresGo] Set bit[27] to 1
CPL Execution – 2	
CPL Execution: • CPL updates System timer (write to each core) • De-assert No Fetch signal to the cores (write to **cpl_cluster_no_fetch** MMR) • Each hart (hardware thread) starts instruction execution • Eventually, M-mode software enables NMI in boot sequence	Registers of interest: • Core MS register (Offset 0xc01): MS_Time register • CPL RC register (Offset 0xc): RstCtrlCplCLNoFetchReg[cplclusternofetch] Set bit[7:0] bits as per enabled cores.

Thermal Management

TjShutdown

If the temperature, as measured by the CPU's internal temperature sensors, exceeds the threshold set for **TjShutdown**, the controller will assert ***o_cluster_interrupts[13]***. In response to this event, the external System Management Controller initiates a transition to minimum functional power and performance, followed by a graceful shutdown, including state saving.

The following steps are performed to handle ***o_cluster_interrupts[13]***:

1. External System Management Controller program PLL Controller Register lowers the Cluster Clock frequency to a minimum functional value.
2. All clock domains (Cluster Clock, Fabric Clock and CPL Clock) are on the same voltage plane. To reduce Voltage, perform a DVFS by:
    a. Reducing Fabric and CPL clock frequency to a minimal operational value.
    b. Driving the Voltage Regulator (VR) to the lower voltage required to support the reduced frequency.

External System Management Controller notifies the OS to save state & shutdown.

Tj Max

If the temperature, as measured by the CPU's internal temperature sensors, exceeds the threshold set for TjMax, the controller will assert ***o_thub_tj_max***.

CAUTION: In response to this event, an immediate VR shutdown is initiated without saving state.

Testbench

The drop includes a testbench that interfaces with and tests a TT-Ascalon Cluster.

Requirements

Verify that your test system meets the following requirements:

- **Supported EDA tools:** See *"Verified EDA Tools" on page 2*.
- **Environment:** See *"Environment Requirements" on page 2*.
- **Container:** See *"Using a Container (Optional)" on page 3* if you are using a container to run your tests.

Building the Testbench

Execute the following command to build the testbench:
make build TARGET=testbench TOOL=<sim> TYPE=<model type>

Where **TYPE** denotes the build type for the target:

- **opt** - for optimized models that build and run quickly.
- **dbg** - for debug models that support dumping waveforms.

Note: The Xcelium debug model requires an existing Verdi installation with $VERDI_HOME properly set in the environment in order to dump waveforms.
If the testbench fails to build, then you can check the full build log located in **$ROOT/design_verification/testbench/build/<sim>/<type>/tt_testbench_<sim>_<type>_build.log**.

Running the Testbench

Running a Testlist

Execute the following commands to run a Testlist (see *"Tests" on page 35* for all included testlists):
make build TARGET=testbench TOOL=<sim> TYPE=<model type> (if not already built) make testlist TARGET=testbench TOOL=<sim> TYPE=<model type> TESTLIST=<testlist>

Using LSF (Optional)

The **design_verification/testbench/run_testlist.sh** script used in the **make** flow can launch tests using LSF but may be subject to varying environment requirements. Add the **bsub** arguments used in **run_testlist.sh** by setting and exporting **TT_EXTRA_LSF_OPTS="<opts>"** before running. The LSF **stdout** will be saved to **$ROOT/design_verification/testbench/runs/<test group>/<test name>_<sim>/lsf.log**. Set **LSF=true** in the **make** command to run the Testlist with LSF.

Running an Individual Test

Execute the following command to run an individual test:
make test TARGET=testbench TOOL=<sim> TYPE=<model type> TESTLIST=<testlist> TEST='<full test file path followed by test args>'

The test executes in a separate run directory under **$ROOT/design_verification/testbench/runs/<TESTLIST>/**

<testname>_<sim>_<model type> with a **run.log**. If a test fails to launch or generate, then all command output will also be available in **$ROOT/design_verification/testbench/runs/<TESTLIST>/ <test>_<sim>_<model_type>/launch.log**. If a test behaves unexpectedly, then please provide the contents of the corresponding runs/ directories to Tenstorrent for debug via your Tenstorrent Customer Success team member.

Checking Test Results

The **testlist** command prints the status of each test as it runs and can also be viewed in result.json in the run directory. You should expect some tests to fail. These tests will start with **expected_fail_**. **design_verification/testbench/test_status.sh** checks the result statuses for all tests under a given directory. By default, **test_status.sh** prints all test failures beneath the current working directory. You can check test results from a different directory by calling **test_status.sh <directory>**. You can also print the status of both passing and failing tests by calling t**est_status.sh <directory> true**. Printing test statuses displays the test name, status, and any applicable failure message. For example:

design_verification/testbench/test_status.sh $ROOT/design_verification/testbench/runs/simple_tests true
Checking status of tests in / testbench dhrystone_100_iter_vcs [Pass]:
Directory: /design_verification/testbench/runs/dhrystone/dhrystone_100_iter_vcs expected_fail_test_1_vcs [Fail]:
Directory: /design_verification/testbench/runs/simple_tests/expected_fail_test_1_vcs Message: "Simulation had error --
Error: Failed stop: Hart 0: write to to-host: 3 "
hello_world_vcs [Pass]:
Directory /design_verification/testbench/runs/simple_tests/hello_world_vcs test_1_vcs [Pass]:
Directory /design_verification/testbench/runs/simple_tests/test_1_vcs

Manually Running a Test Outside of Provided Flows

Execute the **export RUNFILES_DIR=$ROOT/design_verification/testbench** command prior to executing other commands to manually run a test outside of the **make** flow, **run_test.sh**, or **run_testlist.sh** scripts.

Compiling and Running a Custom C Test

The TT-Ascalon Cluster package provides support for compiling custom C tests via the RISC-V GNU toolchain and running them on the TT-Ascalon Cluster testbench. To do this:

1. cd $ROOT/design_verification
2. Install the RISC-V GNU toolchain (See riscv-gnu-toolchain* for environment requirements and installation steps).
3. $ROOT/design_verification/testbench/cc_test_generator -n <name> -s <source test c file>
4. make test TARGET=testbench TOOL=<sim> TYPE=<model type> TESTLIST=<directory name to use> TEST='<test args>'

The TT-Ascalon Cluster includes the following sample C test file: $ROOT/design_verification/tests/ simple_tests/hello_world_debug.c.

Dumping Waveforms

Dump waveforms by setting the TYPE for builds and testlist/test running to dbg, then add

+fsdb_cycle_on=<cycle number> and (optionally) +fsdb_cycle_off=<cycle number> to the testlist(s) that contain the desired test(s) to denote the start and end cycles to dump the FSDB.

Viewing Design Hierarchy and Waveforms

You can use Verdi to open waveforms using the output present in the dbg build directory for a given target by executing the verdi -dbdir <path/to/target_simv.daidir> command.

Tests

The TT-Ascalon cluster includes the following tests:

- Simple: Execute the following command to run a series of short tests as a sanity check. This includes a test that is expected to fail:

make testlist TOOL=<sim> TARGET=testbench TESTLIST=simple

- Benchmark: This testlist contains only a dhrystone test. Execute the following command:

make testlist TOOL=<sim> TARGET=testbench TESTLIST=benchmark

- Compatibility: This testlist containing a RISC-V compatibility testlist that includes thousands of tests. Execute the following command:

make testlist TOOL=<sim> TARGET=testbench TESTLIST=compatibility

- Feature: This testlist tests TT-Ascalon-specific RISC-V features. Execute the following command:

make testlist TOOL=<sim> TARGET=testbench TESTLIST=feature

Integration Targets

The TT-Ascalon Cluster includes two versions of the simulation and physical design target for the TT-Ascalon Cluster:

- simulation with defines and configuration for use with simulators
- physical_design with defines and configurations for use with synthesis tools.

Requirements

Verify that your integration system meets the following requirements:

- Supported EDA tools: See “Verified EDA Tools” on page 2.
- Environment: See “Environment Requirements” on page 2.
- Container: See “Using a Container (Optional)” on page 3 if you are using a container to run your tests.

Building Integration Targets

Execute the following command to build integration targets:
make build TARGET=<target> TOOL=<sim> TYPE=<model type>

Where:

- TARGET denotes which target to build for:
- simulation
- physical_design
- TYPE denotes the target build type:
- opt is for optimized models that build and run quickly.
- dbg is for debug models that support dumping waveforms.

If the target fails to build, then you can find the full build log in $ROOT/<target>/build/<sim>/<type>/tt_cluster_<sim>_<type>build.log.

Linting Integration Targets

Execute the following command to lint your integration target:
make lint TOOL=<tool>

The TOOL variable denotes which linting tool to use (spyglass).
The lint tool only runs on the physical design target; the TARGET option is not used.
You can find Spyglass reports in $ROOT/cluster/physical_design/cluster_lint_spyglass/.

CDC Constraints

CDC constraints for use with Synopsys VC Static clock domain crossing tools are provided under $ROOT/cluster/physical_design/cdc/.

Whisper

Whisper is a reference RISC-V model. The Whisper executable is included in design_verification/whisper/ whisper. The provided TT-Ascalon Cluster testbench natively incorporates whisper for co-simulation checking, but Whisper can also run standalone binaries, as described in the Whisper User Guide (Markdown format) located at design_verification/whisper/README.md.

Linux

design_verification/whisper/linux/ provides a Linux Whisper image. Execute the following commands to boot Linux with Whisper:
$ROOT/design_verification/whisper/whisper --configfile $ROOT/design_verification/whisper/linux/ whisper.json --target $ROOT/design_verification/whisper/linux/bootrom.elf --target $ROOT/design_verification/whisper/linux/fw_payload.elf --fromhost 0x70000000 --tohost 0x70000008 -- quitany --raw

RTOS

design_verification/whisper/rtos/ provides a binary RTOS for use with Whisper. Execute the following commands to run RTOS with Whisper:
$ROOT/design_verification/whisper/whisper --configfile $ROOT/design_verification/whisper/rtos/ whisper.json -s 0x10000 $ROOT/design_verification/whisper/rtos/bootrom.elf $ROOT/design_verification/whisper/rtos/RTOSDemo64.axf

RTOS boots in less than a second, prints the message Hello FreeRTOS!, and then spins waiting for a timer interrupt. Press [CTRL]+[C] to stop the run. You can also configure Whisper to send a timer interrupt to RTOS every 20 microseconds by executing the following commands:

$ROOT/design_verification/whisper/whisper --configfile $ROOT/design_verification/whisper/rtos/ whisper.json -s 0x10000 $ROOT/design_verification/whisper/rtos/bootrom.elf $ROOT/design_verification/whisper/rtos/RTOSDemo64.axf –alarm 20

This causes RTOS to keep sending a message to the sample application. The output will indicate sent and received messages. Press [CTRL]+[C] to stop the run.